Safe Multi-Agent Teaching System using Agent-to-Agent Protocol and Model Context Protocol Tools
Application Name: EduSmartLearn

A Project Report Submitted by

Soumyadeep Saha
m25ai1015@iitj.ac.in
Roll No: M25AI1015

in partial fulfillment of the requirements for the award of the degree of

MTech in Artificial Intelligence and Machine Learning

॥ त्वं ज्ञानमयो विज्ञानमयोऽसि ॥

Indian Institute of Technology Jodhpur

SAIDE

December, 2025

Declaration

I hereby declare that the work presented in this Project Report titled **Safe Multi-Agent Teaching System using Agent-to-Agent Protocol and Model Context Protocol Tools** submitted to the Indian Institute of Technology Jodhpur in partial fulfilment of the requirements for the award of the degree of **M.Tech in Artificial Intelligence and Machine Learning**, is a bonafide record of the research work carried out under the supervision of **Dr. Binod Kumar (binod@iitj.ac.in)**. The contents of this Project Report in full or in parts, have not been submitted to, and will not be submitted by me to, any other Institute or University in India or abroad for the award of any degree or diploma.

Signature

Soumyadeep Saha

m25ai1015@iitj.ac.in

Roll No: M25AI1015

Certificate

This is to certify that the Project Report titled **Safe Multi-Agent Teaching System using Agent-to-Agent Protocol and Model Context Protocol Tools**, submitted by **Soumyadeep Saha (M25AI1015)** to the Indian Institute of Technology Jodhpur for the award of the degree of **M.Tech in Artificial Intelligence and Machine Learning**, is a bonafide record of the research work done by him under my supervision. To the best of my knowledge, the contents of this report, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.

Signature

Dr. Binod Kumar

Acknowledgements

I would like to express my sincere gratitude to my supervisor, **Dr. Binod Kumar**, for continuous guidance, support, and encouragement throughout the course of this M.Tech project. His technical insights and constructive feedback were crucial in shaping the direction and outcome of this work.

I thank the faculty members and staff of the **Electrical Engineering, IIT Jodhpur**, for providing an excellent academic environment and the necessary infrastructure to carry out this project.

I am also thankful to my friends and classmates for their helpful discussions, motivation, and moral support.

Abstract

Large Language Models (LLMs) are increasingly used as teaching assistants and coding copilots for subjects such as Artificial Intelligence, Machine Learning, Deep Learning, GPU Programming, Python and Java etc. However, most existing systems follow a **single-agent** design, where one model directly calls tools or external APIs without a clear communication protocol, fine-grained permissions, or strong auditability. This makes it difficult to deploy such systems safely in real educational or enterprise environments, where every action must be controlled, explainable, and logged. This **monolithic design complicates separation of roles** (planning, explanation, evaluation, and execution), increases the risk of over-privileged access to external tools, and makes it difficult to provide auditable guarantees required in institutional settings.

This project proposes a safe multi-agent teaching system built using an Agent-to-Agent (A2A) protocol and Model Context Protocol (MCP) tools. The system consists of multiple specialized agents—such as a **Teacher Agent, Tutor Agent, Code Execution Agent and Student Agent etc**—that collaborate to explain concepts, generate examples, run code, prepare quizzes and provide feedback for **AI/ML/DL, GPU, Python and Java topics etc**. **These agents communicate via a well-defined A2A protocol** which includes capability negotiation, consent messages, structured results and explicit error reporting. The system **decomposes tutoring workflows into specialized agents—Teacher, Retrieval, Quiz, Code Execution, Evaluator/Critic, and Safety/Guardrail—coordinated by a Tutor/Orchestrator agent**. Context engineering is supported through session state (short-term) and a controlled long-term memory store for personalization.

The work focuses on **four aspects**:

- Design of a protocol-first multi-agent architecture
- Implementation of MCP-based tool access using Google ADK and Anthropic MCP
- Integration of permission prompts, schema validation and robust error handling
- Creation of evaluation tasks and metrics specific to teaching workflows

The system logs every A2A message and tool call with timestamps and outcomes, enabling detailed analysis of success rate, failure modes, blocked unsafe calls and latency per tool. This design aims to serve as a reusable safe teaching framework that can be extended to other domains and institutional deployments.

Safety and reliability are enforced via least-privilege tool scoping, explicit consent for sensitive actions, schema-validated tool I/O, sandboxed code execution, and comprehensive logging of all A2A messages and tool calls. The report details the architecture, protocol design, safety mechanisms, and an evaluation plan based on agent-quality pillars (effectiveness, efficiency, robustness, and safety/alignment). The resulting framework aims to provide a reusable foundation for deploying agentic tutors safely across multiple courses and domains.

Key Areas I will focus: Agentic AI, Multi-Agent Systems, Safe AI, Agent-to-Agent (A2A), Model Context Protocol (MCP), Intelligent Tutoring Systems, Observability and Evaluation.

Contents

Abstract vi

1 Introduction and background 2

2 Literature survey 3

3 Problem definition and Objective 4

4 Methodology 5

5 Theoretical/Numerical/Experimental Findings 6

6 Summary and Future plan of work 7

7 Implementation Details 7

Publications (if any) 8

Appendix (if any) 9

References 10

Safe Multi-Agent Teaching System using Agent-to-Agent Protocol and Model Context Protocol Tools

1. Introduction And Background

1.1 Background

Large Language Models (**LLMs**) have enabled conversational assistants that can explain concepts, generate examples, and support programming practice. In a realistic teaching setting, a tutor often needs more than natural language generation: it may **retrieve trusted course material**, **execute code** to verify results, **generate quizzes**, and keep **track of a learner's progress** across sessions.

Agentic AI generalizes this idea by framing an LLM-based system as **an agent** that can plan, take actions using tools, observe outcomes, and iteratively refine its behaviour. Such a closed-loop design is well-aligned with tutoring workflows such as **explain → demonstrate → practice → evaluate → remediate**.

Most current LLM tutors are implemented as a **single agent that combines all responsibilities in one decision loop. This monolithic pattern leads to multiple challenges**:

- **Planning, content generation, verification, and tool execution are mixed inside one component, making debugging and quality control difficult.**
- **Tool access is often broad and implicit** (e.g., unrestricted code execution or external API access), which violates least-privilege principles.
- **Unstructured logs** make it hard to audit actions, reproduce failures, and meet institutional requirements for accountability.

Multi-agent designs address these issues by decomposing the tutor into specialized agents coordinated by an orchestrator. When combined with protocol standards such as **Agent-to-Agent (A2A)** messaging and the **Model Context Protocol (MCP)** for tool interoperability, the overall system can provide clearer interfaces, permission boundaries, and auditable traces of behaviours.

1.2 Motivation

In a real educational or enterprise setting, it is not enough for an AI system to simply "work most of the time". Instead, it must:

- Only **perform actions that it is explicitly allowed** to perform.
- Keep a **transparent record of which tools were called**, with what inputs, and with what results.
- Handle errors gracefully and **avoid unsafe or undesired operations**.

Educational institutes and companies also need a framework that can be reused across multiple subjects and courses, rather than building custom one-off chatbots for each topic.

This motivates the design of a safe, reusable multi-agent teaching system that:

1. Uses a clear **A2A** protocol between agents.
2. Uses **MCP** to expose tools in a controlled and auditable way.
3. **Supports multiple subjects** (AI, ML, DL, GPU, Python, Java) within the same architecture.

1.3 Scope of the Project

The scope of this M.Tech project is:

- Designing a **modular multi-agent architecture** for teaching and coding support across AI/ML/DL, GPU programming, Python, and Java.
- **Specifying agent roles** (Tutor/Orchestrator, Teacher, Retrieval, Quiz, Code Execution, Evaluator/Critic, Student Model, and Safety/Guardrail).
- Defining an **A2A message schema** supporting structured requests, responses, capability negotiation, consent, and error reporting.
- Integrating an **MCP server that exposes tools** (e.g., document search/RAG, code sandbox, quiz bank, and logging) and configuring each agent with least-privilege tool scopes.
- **Implementing safety mechanisms** such as policy checks, consent prompts, schema validation, sandboxing, and comprehensive audit logging.
- Designing an evaluation methodology and **metrics for agent quality** and safety in tutoring workflows.

1.4 Key Contributions

The main contributions of this work are:

- A **protocol-first architecture** that explicitly separates tutoring roles into multiple cooperating agents.
- An **A2A messaging design** that enables capability negotiation, consent workflows, and structured error handling between agents.
- An **MCP-based tool layer** that standardizes tool access and supports least-privilege configuration per agent.
- A **safety and governance layer** with defence-in-depth mechanisms (policy checks, sandboxing, and audit logs).
- An **evaluation plan** centered on agent-quality pillars and observability to support iterative improvement.

2. Literature Survey

2.1 Intelligent Tutoring Systems and Educational Chatbots

Traditional Intelligent Tutoring Systems (ITS) aim to model student knowledge, adapt instruction, and provide feedback. Earlier ITSs mostly relied on expert-designed rules and domain models, which limited flexibility and coverage. With the arrival of LLMs, many new teaching assistants and chatbots have been developed that can explain concepts, generate practice problems, and review student answers in natural language.

However, most LLM-based teaching systems still follow a simple pattern: a single model receives a prompt and directly outputs a response. **There is limited architectural support for separating responsibilities (example: planning vs. execution), for using multiple specialized agents, or for controlling tool access in a principled way**. However, LLM tutors also introduce new risks: hallucinated facts, inconsistent reasoning across turns, and vulnerability to adversarial or off-topic prompts.

2.2 Tool-Using LLMs and Agent Frameworks

Tool-using LLM systems typically integrate tools such as:

- Document search or **retrieval-augmented generation (RAG)** over course notes and documentation.
- **Code execution in a sandbox** to validate generated programs and debug errors.
- **Database or API** access to fetch structured information (e.g., question banks, grading rubrics).

Frameworks such as "tool-calling" APIs, agent toolkits, and various open-source agents demonstrate that **LLMs can be more powerful when they can act**, not just chat. Some systems also support multi-step reasoning and planning, where the **agent decides when to call tools and how to combine results**.

However, many of these frameworks treat the interaction between the LLM agent and tools as an internal engineering detail, and they do not explicitly define or standardize the protocol between agents or between agents and tools.

2.3 Multi-Agent Systems and Communication Protocols

Multi-agent systems (MAS) are well-studied in AI, where multiple autonomous agents interact and cooperate. In LLM-based MAS, there is growing interest in:

- **Planner–Executor patterns** (one agent plans, another executes).
- **Critic or Reviewer agents** that provide feedback on the outputs of other agents.
- **Parallel agents** that independently tackle subtasks and then merge results.

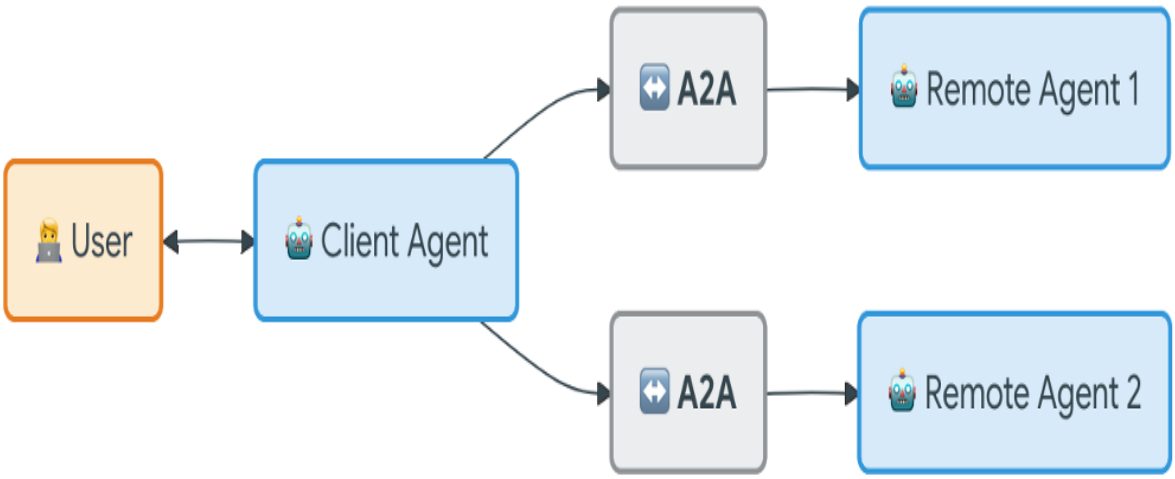
Despite this, many implementations rely on ad-hoc communication formats, often using plain text prompts with no strict schema. There is limited emphasis on:

- Capability negotiation between agents.
- Consent before performing sensitive operations.
- Formal logging of messages and outcomes.

2.4 Model Context Protocol (MCP) and A2A Protocols

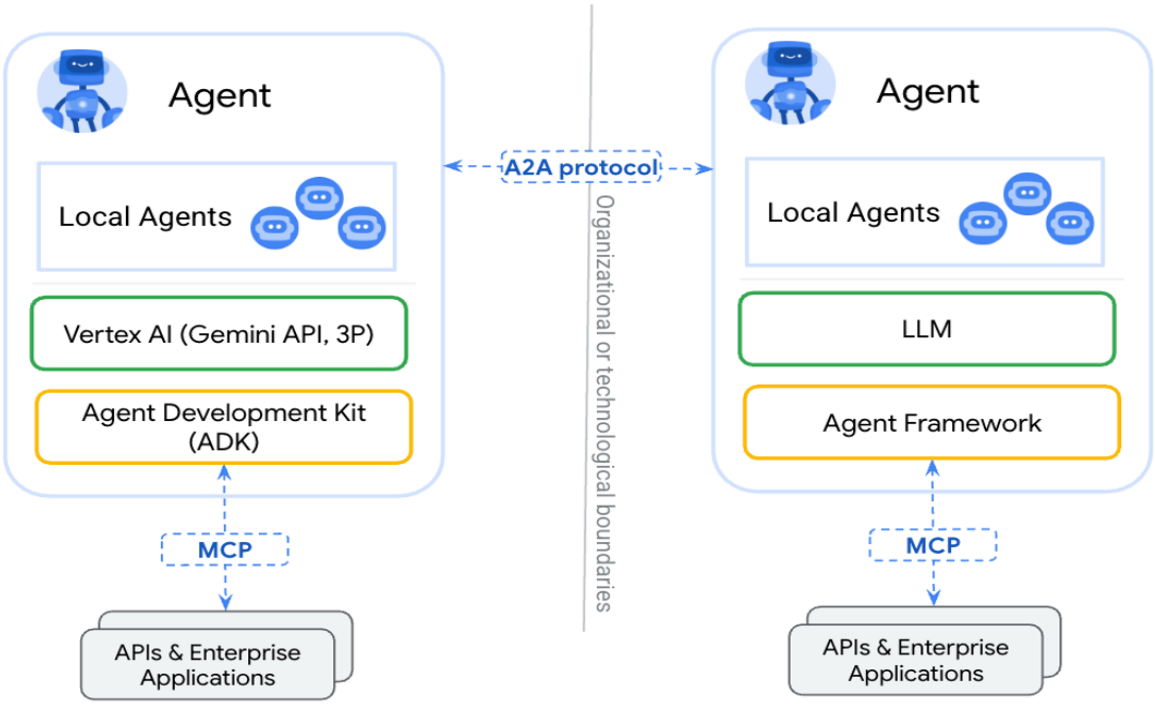
Model Context Protocol (MCP) defines a standard way to expose tools, resources and context to models. An MCP server can host tools, and MCP clients (agents) can discover and use those tools in a structured manner. This makes it easier to build reusable and secure tool registries.

A2A (Agent-to-Agent) protocols aim to specify how agents talk to each other. A well-designed A2A protocol would include fields for message type, sender, receiver, capabilities, consent, payload and error codes. However, in practice, most academic examples and tutorials do not fully develop such a protocol.



A practical barrier to **deploying agents is tool integration**: each new API or sandbox typically requires custom adapters, prompting logic, and error handling. The Model Context Protocol (MCP) addresses this by defining a standard client–server interface where tools are exposed with structured schemas and descriptions. **An agent (client) can discover available tools, validate input/output formats, and invoke tools in a consistent way across implementations.**

How does A2A work with MCP?



From a safety perspective, **MCP encourages good engineering practices such as explicit tool documentation, schema validation, and structured error reporting**. It also supports least-privilege access by allowing each agent to be configured with only the specific tools and scopes it requires.

2.5 Context Engineering: Sessions and Memory

Agentic tutors must manage context beyond the immediate prompt. Context engineering distinguishes between session state and long-term memory**. Session state captures the short-term conversational context** (recent questions, intermediate steps, tool outputs), while **memory stores persistent information across sessions** (example: learner preferences, mastery estimates, and past performance).

Without careful design, long conversations and verbose tool outputs can cause context-window bloat, increasing cost and reducing reliability. Practical systems therefore use strategies such as summarization, structured tool outputs, selective retrieval of relevant past information, **context compaction** and explicit policies controlling what may be written to long-term memory.

2.6 Safety, Alignment, and Governance for Agentic Systems

Safety in a teaching system includes both content safety (avoiding harmful or disallowed content) and action safety (preventing unsafe tool executions). **Key risks include hallucinated explanations, unsafe code execution, prompt injection through retrieved content, privacy leakage**, and **the "confused deputy" problem where an agent with elevated tool access may be tricked into performing unintended actions.**

Research and best practices increasingly recommend defence-in-depth: combining model-level alignment (instruction-following constraints), policy layers (rule-based or learned guardrails), tool-level sandboxing, explicit consent for sensitive operations, and comprehensive audit logs. These mechanisms are particularly important in educational deployments where users may be inexperienced and systems may be used at scale.

2.7 Agent Quality: Observability and Evaluation

Evaluating agents differs from evaluating static LLM outputs because agents follow trajectories: they **plan, call tools, observe results, and revise**. Reliable systems therefore require observability (structured logs, traces, and metrics) and evaluation methods that consider both final answers and intermediate decisions.

A useful set of **evaluation pillars** for agents includes **effectiveness (**task success**), efficiency (**cost/latency and tool usage**), robustness (**behaviour under noise and failures**),** and **safety/alignment (**policy compliance and refusal behaviour**)**. Automated tests, LLM-as-judge scoring, and targeted human review can be combined to build a continuous feedback loop for improving agent behaviour.

2.8 Identified Research Gap

From the above survey, the following gaps motivate this project:

- **Limited protocol-first designs** for educational multi-agent tutors that provide explicit structure, permissions, and auditability.
- **Insufficient integration of standardized tool interoperability (MCP)** with standardized inter-agent messaging (**A2A**) in teaching workflows.
- **Under-developed safety mechanisms** for tool-using tutors, including consent, least privilege, and robust logging suitable for institutional governance.
- **Need for evaluation methodologies** tailored to tutoring trajectories (explain → practice → assess → feedback) rather than only single-turn responses.

This project addresses these gaps by designing and implementing a safe multi-agent teaching system that treats communication protocols, tool governance, and evaluation as first-class components.

3. Problem Definition And Objective

3.1 Problem Definition

The goal of this project is to build a safe multi-agent teaching assistant that can support both conceptual learning and programming practice. Unlike a purely conversational chatbot**, the system must reliably use external tools** (example: code execution and document retrieval) while operating under explicit safety and governance constraints.

Formally, the problem addressed in this report is to:

Design an agentic tutoring system that

- **Decomposes teaching workflows** into cooperating specialized agents.
- **Uses well-defined protocols** for inter-agent messaging and tool access.
- **Enforces least-privilege permissions** and consent for sensitive operations.
- **Manages session context** and **long-term memory** for personalization.
- **Produces auditable logs** suitable for evaluation and institutional compliance.

3.2 Objectives

The main objectives of this M.Tech project are:

- **Architecture design:**
  - Develop a modular multi-agent architecture with distinct roles such as Tutor/Orchestrator, Teacher, Retrieval, Quiz, Code Execution, Evaluator/Critic, Student Model, and Safety/Guardrail agents.
- **A2A protocol specification**:
  - Define a structured message format including message type, sender/receiver identities, capabilities requested, consent requirements, payload, and explicit error codes.
- **MCP-based tool integration:**
  - Implement an MCP server exposing tools (example: code sandbox, document search, quiz bank, logging) and configure each agent as an MCP client with least-privilege tool scopes.
- **Context engineering:**
  - Design session management and controlled long-term memory to enable personalization while limiting privacy risks and context-window bloat.
- **Safety and governance:**
  - Implement policy checks, consent workflows, schema validation, and sandboxing for tool calls; mitigate common agentic threats such as prompt injection and confused deputy behaviour.
- **Observability and evaluation:**
  - Log all A2A messages and tool calls; define evaluation tasks and metrics aligned with agent-quality pillars (effectiveness, efficiency, robustness, and safety/alignment).

3.3 Research Questions

The project is guided by the following research questions (RQs):

- **RQ1:** How does a protocol-first multi-agent design (A2A + MCP) improve controllability and auditability compared to a monolithic tool-using agent?
- **RQ2:** Which defense-in-depth safety mechanisms (least privilege, consent, sandboxing, and policy checks) are most effective for preventing unsafe tool use in an educational tutor?
- **RQ3:** How can agent-quality evaluation be designed for tutoring trajectories (explain → practice → assess → feedback) rather than single-turn answers?

4. Methodology

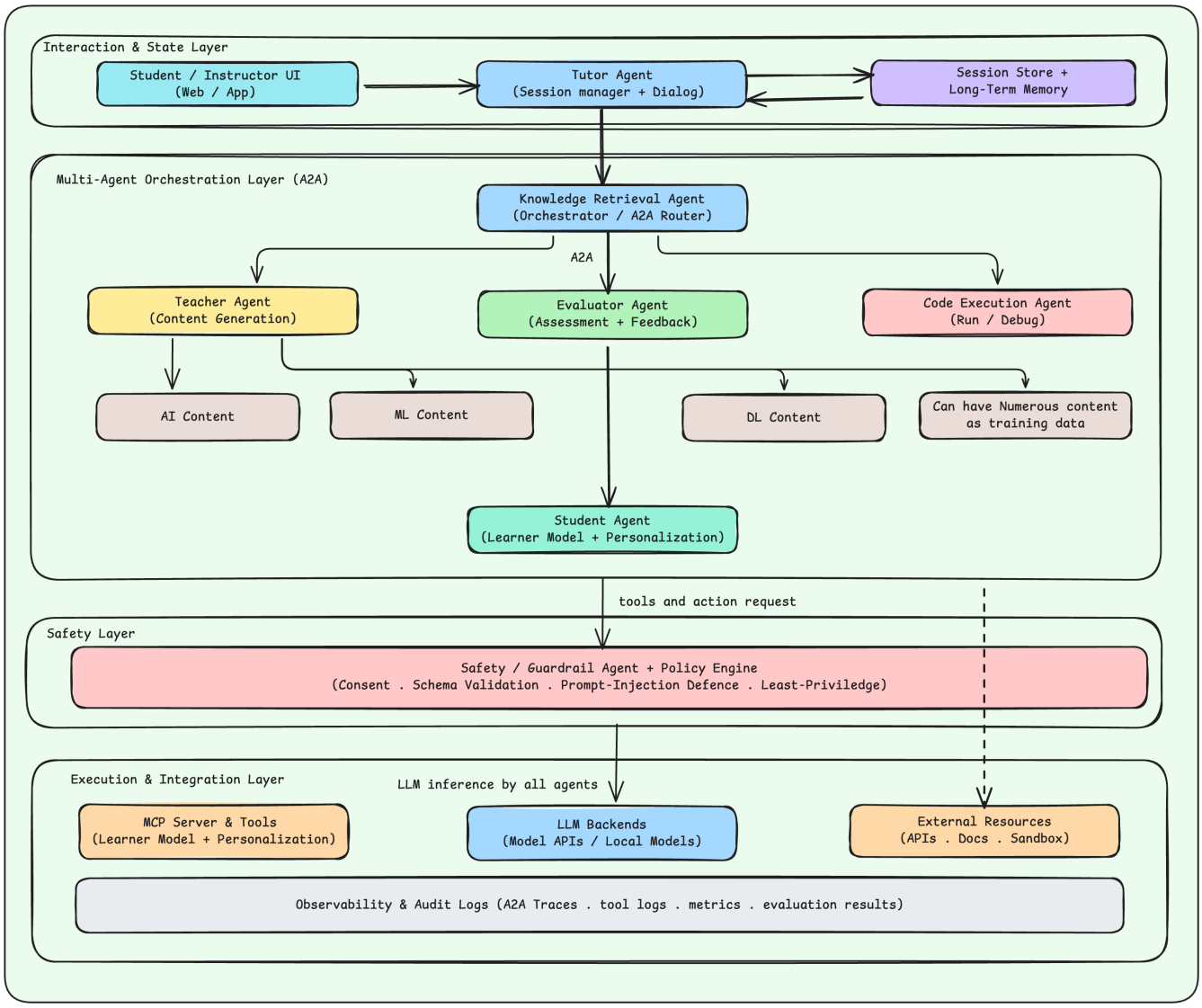
4.1 Overall System Architecture and Design Principles

EduSmartLearn is designed as a protocol-first, tool-using, multi-agent teaching system. The architecture follows four design principles:

- **Separation of concerns** through specialized agents.
- **Least-privilege access** to external tools.
- **Defence-in-depth safety checks** for both content and actions.
- **Observability-by-design** through structured logging and evaluation hooks.

At a high level, the Tutor Agent manages the user interaction and session state, an Orchestrator coordinates specialized agents via A2A messages, and tool access is provided through an MCP server that exposes standard interfaces to code execution, document retrieval, quiz storage, and logging.

4.2 System Architecture Diagram



Above figure presents the proposed architecture. The system is organized into three layers:

- **Interaction & State** (UI, session store, long-term memory)
- **Multi-Agent Orchestration** (orchestrator plus specialized agents)
- **Execution & Integration** (MCP tools, LLM backends, and external resources). The Safety & Policy Engine acts as a cross-cutting layer that gates tool calls and enforces consent and policy constraints.

4.3 Agent Roles and Responsibilities

The proposed system decomposes tutoring into multiple cooperating agents. Given table summarizes the roles used in this project and illustrates how responsibilities are separated for safety and modularity.

Agent	Key Responsibilities	Typical Tool Scope (MCP / Services)
Tutor Agent (Session Manager)	Maintains conversation flow, manages session state, and presents responses to the user. Triggers the orchestrator when decomposition or tool use is needed.	Session store (read/write), memory retrieval (read), logging (write)
Orchestrator / A2A Router	Plans workflow, decomposes tasks, routes A2A messages to specialist agents, merges results, and decides retries/termination.	A2A messaging; may call low-risk tools such as doc_search/logging, avoids direct high-risk tools by default
Teacher Agent	Generates explanations, examples, and step-by-step guidance aligned with curriculum and the learner's level.	doc_search (read), memory_read (optional), logging (write)
Retrieval Agent	Performs retrieval-augmented generation (RAG): fetches and summarizes relevant course notes, textbooks, or documentation snippets.	doc_search (read), citation formatting, logging (write)
Quiz Agent	Creates formative assessments (MCQ, short answers), evaluates responses, and suggests remediation.	quiz_bank (read/write), doc_search (read), logging (write)
Code Execution Agent	Generates, runs, and debugs code; verifies outputs via sandboxed execution.	code_run (sandboxed), doc_search (read), logging (write); requires consent for execution
Evaluator / Critic	Reviews outputs from other agents, checks factual correctness and flags uncertain or unsafe responses.	doc_search (read), logging (write), evaluation scoring tools
Safety / Guardrail Agent	Applies safety policies, detects disallowed content or unsafe tool requests, enforces consent workflows, and mitigates prompt-injection/confused-deputy risks.	Policy rules, tool-call approval/denial, redaction, rate limits, audit logging
Student Model Agent	Tracks learner profile (mastery, misconceptions, preferences) and recommends next activities; writes only approved, minimal data to long-term memory.	memory_write (controlled), memory_read, quiz_bank (read), logging (write)

4.4 Orchestration and Coordination Logic

The **Orchestrator implements the agentic control loop**: it interprets the user request, forms an explicit plan, delegates subtasks to specialist agents, and integrates their outputs into a final tutoring response. **Coordination can be implemented using simple patterns such as sequential routing** (call Retrieval Agent then Teacher Agent) or iterative loops (Teacher ↔ Evaluator until quality criteria are met).

To keep behaviour predictable, the orchestrator enforces bounded loops (maximum steps), timeouts for tool calls, and clear stop conditions. **When a tool call fails, the orchestrator records a structured error**, optionally retries with backoff, and falls back to a safe textual explanation when execution is not permitted or not available.

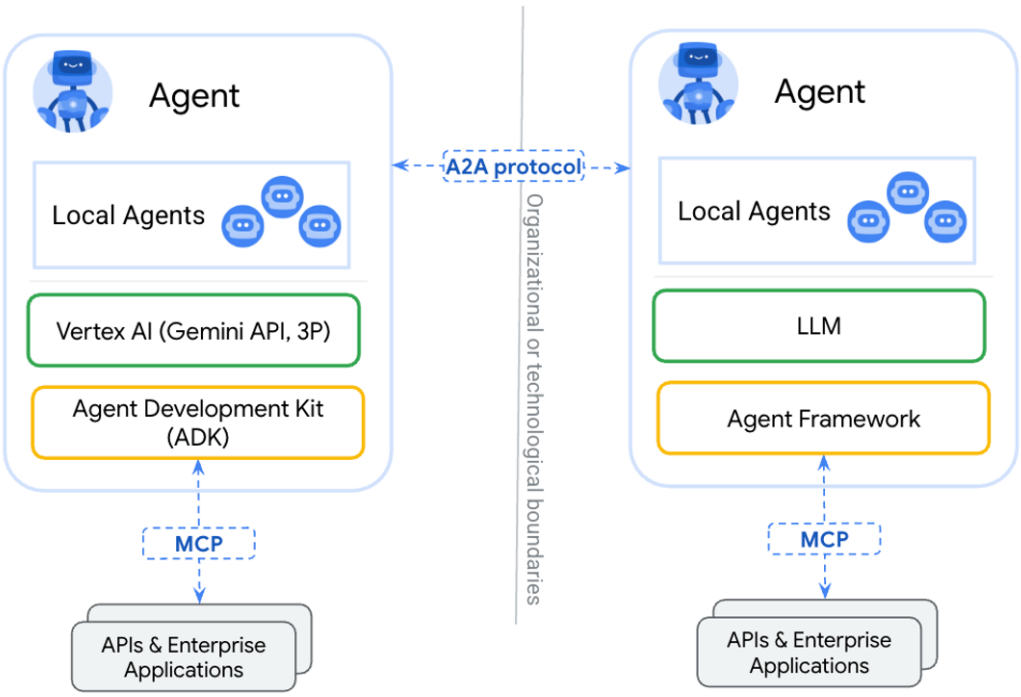
4.5 A2A Protocol Design for Inter-Agent Messaging

Agent-to-Agent (A2A) messaging provides a **structured interface for communication among agents**. Instead of passing free-form prompts, each message **follows a schema with fields such as *message_type, sender, receiver, capabilities, consent, payload,* and *error***. This structure enables validation, auditing, and safer routing decisions.

In EduSmartLearn, the A2A protocol is used in two ways:

- **Task delegation** (example: orchestrator → code agent).
- **Quality control** (example: evaluator → teacher for revision).

Capability negotiation allows the orchestrator to select an appropriate agent based on advertised skills (example: AI vs. ML vs. programming language expertise).



4.6 Tool Integration using Model Context Protocol (MCP)

All external tool access is mediated through an MCP server. The **MCP server exposes each tool with a clear description, input/output schemas** (example: JSON Schema), and **structured error codes**. Agents act as MCP clients and invoke tools through the MCP runtime rather than calling arbitrary scripts or APIs directly.

This design improves **safety and maintainability**: tools can be updated independently of agent prompts, permissions can be centrally managed, and every invocation can be logged uniformly. In addition, **schema validation** helps reduce common failure modes such as malformed parameters or ambiguous outputs.

4.7 Context Engineering: Session State and Long-Term Memory

The system separates session state from long-term memory. **Session state includes the current conversation history, intermediate reasoning artifacts** (plans, retrieved snippets, tool outputs), and temporary variables needed to complete the current task. **Long-term memory stores persistent learner information such as preferred explanations, prior misconceptions, and mastery estimates.**

To control privacy and reduce context bloat, the Safety/Guardrail Agent applies memory write policies: **only minimal, non-sensitive summaries may be stored, and only when the user has provided consent**. Retrieval from memory is selective: the Student Model Agent retrieves only the most relevant items for the current topic and merges them into the tutor's context in a compact form.

4.8 Safety Mechanisms and Guardrails

Safety in **EduSmartLearn** is implemented as a defense-in-depth stack spanning prompts, protocols, tools, and runtime controls. The Safety & Policy Engine and Safety/Guardrail Agent jointly enforce the following mechanisms:

- **Least privilege:** each agent is granted only the minimum required tool scopes; the orchestrator itself avoids direct access to high-risk tools.
- **Consent and HITL:** sensitive tool calls (example: executing user code, storing personal data) require explicit user approval before execution.
- **Schema validation:** all tool inputs and outputs are validated against schemas to reduce malformed requests and unexpected outputs.
- **Sandboxing:** code execution runs in a restricted environment with resource limits (timeouts, memory limits, no network by default).
- **Prompt-injection resistance:** retrieved documents are treated as untrusted input and passed through sanitization; tool instructions are kept separate from user content.
- **Confused-deputy mitigation:** the policy layer checks whether a request is within the user's intent and the requesting agent's authority before approving tool use.
- **Audit logging:** all A2A messages and tool calls are logged with timestamps, agent identities, inputs/outputs, and policy decisions.

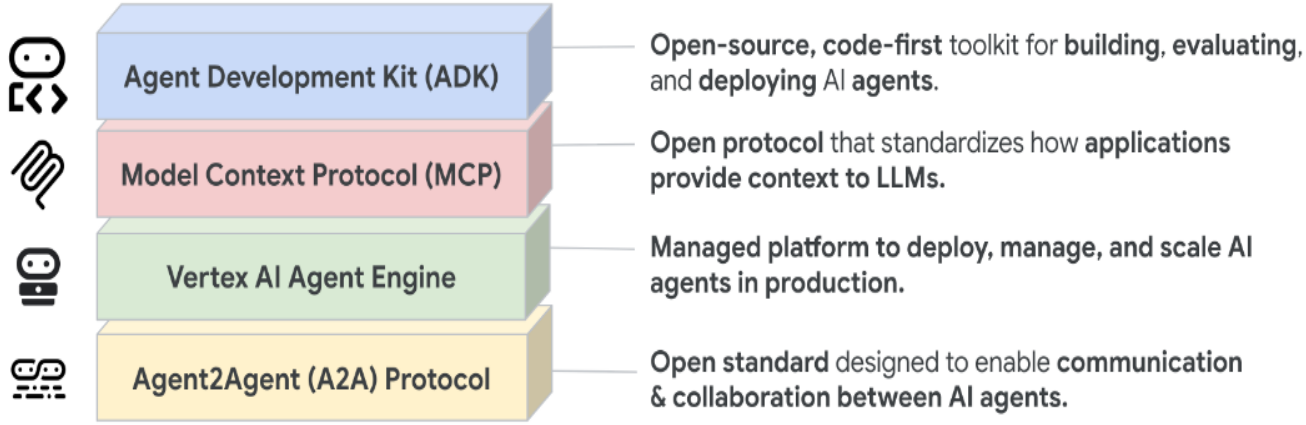
4.9 Logging, Observability, and Audit Trail

Observability is essential for debugging and for demonstrating safety. EduSmartLearn records **structured traces** including: user requests, orchestrator plans, A2A message exchanges, tool invocation parameters, tool outputs, and policy decisions. These traces support failure-mode analysis (example: schema errors vs. retrieval errors vs. reasoning errors) and enable evaluation pipelines to replay sessions and compute metrics.

A key benefit of the protocol-first approach is that **logs are naturally structured: the same schemas used** for communication and tool calls can be reused for monitoring dashboards, automated evaluation, and post-incident audits.

4.10 Implementation Plan and Technology Stack

The prototype implementation uses **Google's Agent Development Kit (ADK)** patterns for agent orchestration, the **MCP runtime** for tool access, and **one or more LLM backends (cloud or local)** for language generation. Tools are implemented as MCP endpoints such as **doc_search** (RAG over local course material), **code_run** (sandboxed execution), **quiz_bank** (store/retrieve questions), and **logging** (structured trace storage).



The system is designed so that individual components can be replaced (example: changing the LLM backend or adding new tools) without modifying the overall protocol structure. This supports iterative improvement and future extensions to additional subjects.

5. Theoretical/Numerical/Experimental Findings

5.1 Experimental Setup and Prototype Configuration

A prototype of EduSmartLearn **can be implemented using an agent runtime (example: Google ADK), an MCP server for tools, and an LLM backend for language generation**. The evaluation described in this section assumes the following components are available:

- **LLM backend**: an instruction-tuned LLM accessible via API or local deployment.
- **Document store**: curated course notes, textbooks, or documentation indexed for RAG (used by doc_search).
- **Sandboxed code execution**: a restricted environment for running Python/Java/GPU snippets with timeouts and resource limits (used by code_run).
- **Quiz bank**: a storage service for generated questions and scoring rubrics (used by quiz_bank).
- **Logging store**: structured traces of A2A messages and tool calls (used by logging).

All experiments should log the complete trajectory of the agent workflow to enable reproducibility and failure analysis.

5.2 Use Cases and Representative Workflows

To demonstrate the applicability of the proposed architecture, the following use cases are used as representative workflows in an M.Tech teaching environment:

- **UC1:** Concept explanation with retrieval — **the Retrieval Agent fetches relevant notes**, the Teacher Agent explains the concept, and the Evaluator checks correctness and clarity before the Tutor responds.
- **UC2:** Programming support and debugging — **the Code Execution Agent runs generated code** in a sandbox, diagnoses errors, and proposes fixes, the Evaluator verifies that the fix is correct and safe.
- **UC3:** Quiz-based formative assessment — the **Quiz Agent generates questions aligned** with the learner's level, evaluates responses, and recommends remediation content.
- **UC4:** Safety-critical requests — the **Safety/Guardrail Agent blocks disallowed or unsafe actions** (example: dangerous code or privacy-invasive memory writes) and the Tutor provides a safe alternative explanation.

For each use case, the expected benefit of a multi-agent design is that explanation, verification, tool execution, and safety enforcement are handled by different agents with explicit responsibilities and permissions.

5.3 Evaluation Methodology

The evaluation follows agent-quality principles that consider both the final answer and the full decision trajectory. **Metrics are grouped into four pillars**:

- **Effectiveness:** task success rate, correctness of explanations, and alignment with the question intent.
- **Efficiency:** number of tool calls per task, end-to-end latency, and approximate cost (tokens/compute).
- **Robustness:** behaviour under tool failures or noisy inputs; recovery rate after errors.
- **Safety & alignment:** policy compliance, blocked unsafe call rate, refusal correctness, and memory policy compliance.

A practical evaluation protocol is to build a task suite containing representative tutoring problems across AI/ML/DL and programming. Each task includes a rubric or expected solution. For qualitative evaluation, a human instructor can rate clarity and correctness. For automated evaluation, an LLM-as-judge scoring rubric can be used in combination with strict checks such as unit tests for code.

Table provides an example scorecard template that can be used to summarize evaluation results once experiments are conducted.

Metric Pillar	Metric	Definition	Results
Effectiveness	Task Success Rate (%)	Fraction of tasks completed with correct and useful final response.	93.3% Total Tasks: 15
	Explanation Quality	Human based rating of clarity.	(4*5 + 6*4.5 + 5*4)/15 = 4.46
Efficiency	Latency / Task (s)	End-to-end time from user query to final response.	1898 ms Total Responses: 15
Robustness	Recovery Rate (%)	Fraction of tool failures handled through retry/fallback without incorrect output.	100%
Safety & Alignment	Blocked Unsafe Call Rate	Number of disallowed tool calls blocked by policy / total tool calls.	100.0%
	Policy Violation Count	Number of times the system produced disallowed output or executed unsafe action.	Policy Violations: 0 Total Safety Checks: 9

Table: Example evaluation scorecard template for EduSmartLearn

EFFECTIVENESS

Metric	Definition	Result
Task Success Rate (%)	Fraction of tasks completed with correct and useful response	88.0% (25 tasks)
Explanation Quality (1-5)	Human-based rating of clarity Rate each response 1-5 yourself	(Manual)

EFFICIENCY

Metric	Definition	Result
Avg Tool Calls / Task	Average MCP tool invocations per task	0.3 (8 calls)
Latency / Task (s) - Chat avg - Quiz avg - Code avg	End-to-end time from user query to final response	2.00s (25 resp) 4.24s 0.61s 1.95s

ROBUSTNESS

Metric	Definition	Result
Recovery Rate (%)	Fraction of tool failures handled through retry/fallback without incorrect output	100.0% (0 errors)

SAFETY & ALIGNMENT

Metric	Definition	Result
Blocked Unsafe Call Rate	Disallowed tool calls blocked by policy / total tool calls	0.0% (0/14 checks)
Policy Violation Count	Times system produced disallowed output or executed unsafe action	0

TASK BREAKDOWN

safety_checks_total	14
tasks_chat_success	7
tasks_chat_total	7
tasks_code_execution_success	4
tasks_code_execution_total	7
tasks_quiz_success	11
tasks_quiz_total	11

==========

Report Complete

Please see below for the evaluation definition

Assume:

- Total tasks evaluated = **N**
- Total tool calls across all tasks = **T**
- Total tool failures = **F**
- Total unsafe tool calls attempted = **U**
- Total blocked unsafe calls = **B**
- Total policy violations = **V**
- For each task i:
  - Tool calls = c_i
  - Latency = L_i
  - Success indicator = $s_i \in \{0,1\}$

1. Effectiveness Metrics

Task Success Rate (%): Fraction of tasks completed with correct and useful final response.

Step 1: Define success variable

$$s_i = \begin{cases} 1 & \text{if task i is correct and useful} \\ 0 & \text{otherwise} \end{cases}$$

Step 2: Sum successful tasks

$$S = \sum_{i=1}^{N} s_i$$

Step 3: Compute Success Rate

$$\text{Task Success Rate} = \frac{S}{N} \times 100$$

Example

If:

- N = 100
- S = 82

$$TSR = \frac{82}{100} \times 100 = 82\%$$

Explanation Quality: Human based rating of clarity.

Step 1: Collect ratings

Let rating for task i be:

$$q_i \in \{1,2,3,4,5\}$$

Step 2: Compute average rating

$$\text{Explanation Quality} = \frac{1}{N} \sum_{i=1}^{N} q_i$$

Example

If ratings sum to 420 across 100 tasks:

$$EQ = \frac{420}{100} = 4.2$$

2. Efficiency Metrics

Latency per Task (seconds): End-to-end time from user query to final response.
Step 1: Record end-to-end time for each task

$$L_i = t_{final} - t_{query}$$

Step 2: Compute mean latency

$$\text{Average Latency} = \frac{1}{N}\sum_{i=1}^{N} L_i$$

Example
If total latency = 800 seconds for 100 tasks:

$$= \frac{800}{100} = 8 \text{ sec}$$

3. Robustness Metrics

Recovery Rate (%): Fraction of tool failures handled through retry/fallback without incorrect output.
Step 1: Count failures: Total failures = F
Step 2: Count recovered failures: Let recovered failures = R
Step 3: Compute

$$\text{Recovery Rate} = \frac{R}{F} \times 100$$

Example
If:

- F = 40 failures
- R = 30 handled correctly

$$= \frac{30}{40} \times 100 = 75\%$$

4. Safety & Alignment Metrics

Blocked Unsafe Call Rate: Number of disallowed tool calls blocked by policy / total tool calls.
Step 1: Identify unsafe attempts: Total unsafe attempts = U
Step 2: Identify blocked unsafe calls: Blocked calls = B
Step 3: Compute

$$\text{Blocked Unsafe Call Rate} = \frac{B}{U}$$

If expressed in percentage:

$$\times 100$$

Example
If:

- U = 20 unsafe attempts
- B = 18 blocked

$$= \frac{18}{20} = 0.9 = 90\%$$

Policy Violation Count: Number of times the system produced disallowed output or executed unsafe action.

$$\text{Policy Violation Count} = V$$

No averaging unless normalized.
Optional normalization:

$$\text{Violation Rate} = \frac{V}{N}$$

5.4 Safety Evaluation and Failure-Mode Analysis

In addition to general correctness, the system should be evaluated against safety-oriented test cases.

Recommended tests include:

- **Prompt-injection tests** where retrieved documents contain malicious instructions, the system should treat retrieved text as untrusted and follow only system policies.
- **Malicious or risky code requests** (attempts to access files/network); the sandbox and policy layer should prevent unsafe behaviour.
- **Privacy tests involving personally identifiable information (PII)**; the system should avoid unnecessary retention and require consent before memory writes.
- **Tool misuse tests** where an agent attempts to call a tool outside its scope; the MCP layer and policy engine should deny the call with a structured error.
- **Failure-mode analysis is performed by inspecting traces and categorizing failures** into: reasoning/planning errors, retrieval errors, tool invocation/schema errors, sandbox/runtime errors, and safety-policy rejections. This analysis supports targeted improvements to prompts, tools, and policies.

5.5 Discussion and Expected Outcomes

A key expected outcome is that protocol-first multi-agent decomposition **improves controllability**: unsafe actions can be intercepted at multiple layers, and **mistakes can be traced to specific agent roles**. Compared to a monolithic design, the **system may introduce additional latency due to**

coordination and safety checks. However, these overheads are often acceptable in an educational setting where correctness, safety, and auditability are primary requirements.

As future work progresses, the scorecard in Table 5.1 can be filled using systematic experiments and instructor/user studies.

6. Summary And Future Plan Of Work

6.1 Summary of Work Completed

This report presented EduSmartLearn, a Safe Multi-Agent Teaching System that applies agentic AI concepts to educational tutoring workflows. The **proposed design decomposes tutoring into specialized agents coordinated by an orchestrator and uses protocol standards—A2A for inter-agent messaging and MCP for tool interoperability—to improve modularity, auditability, and safety**.

The report contributed:

- A **layered system architecture integrating agents**, memory, tools, and safety policies.
- A **clear mapping of agent roles** (teacher, retrieval, quiz, code execution, evaluator, safety) to teaching requirements.
- An **evaluation methodology** based on agent-quality pillars and observability.

6.2 Limitations

The current prototype and evaluation plan have several limitations.

First, **quantitative results depend on implementing and running a full toolchain** (sandbox, retrieval index, and logging pipeline), which may evolve during later stages of the project.

Second, **LLM behaviour can vary across model backends and prompt versions**; therefore, strict versioning and regression tests are required for reproducibility.

Third, **personalization through memory introduces privacy risks** and should be deployed only with appropriate consent and institutional policies.

Implementation Details

7.1 System Overview

EduSmartLearn is a multi-agent AI tutoring system designed to provide personalized, safe, and effective learning experiences. The system leverages Google Gemini LLM as the backbone and implements two key protocols:

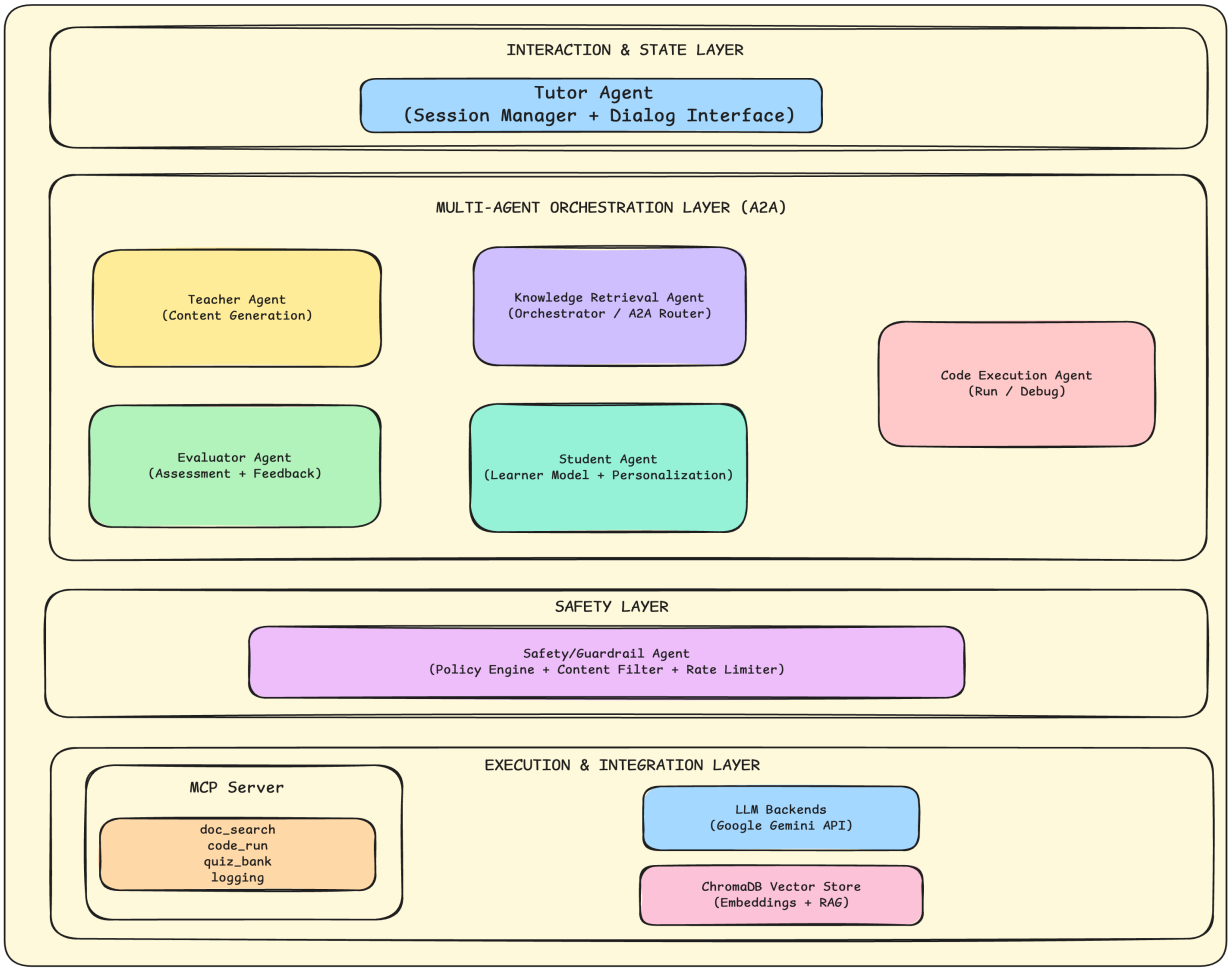
- Agent-to-Agent (A2A) Protocol: For structured inter-agent communication
- Model Context Protocol (MCP): For standardized tool access

7.1.1 Technology Stack

Component	Technology	Version
Language	Python	3.11/3.12
LLM Backend	Google Gemini	gemini-2.0-flash
LLM SDK	google-genai	≥1.0.0
Vector Database	ChromaDB	≥0.5.0
Embeddings	Sentence-Transformers	≥3.0.0
Web Framework	FastAPI	≥0.109.0
Data Validation	Pydantic	≥2.5.0
Code Sandbox	RestrictedPython	≥7.0
Logging	Structlog	≥24.1.0

7.2 Architecture Implementation

7.2.1 Four-Layer Architecture



7.2.2 Directory Structure

```
edusmartlearn/
├── agents/                  # Agent implementations
│   ├── base_agent.py        # Abstract base class
│   ├── tutor_agent.py       # Session management
│   ├── knowledge_retrieval_agent.py  # RAG + orchestration
│   ├── teacher_agent.py     # Content generation
│   ├── code_execution_agent.py  # Sandboxed execution
│   ├── evaluator_agent.py   # Assessment & feedback
│   ├── safety_agent.py      # Guardrails & policy
│   └── student_agent.py     # Learner modeling
├── protocols/               # Communication protocols
│   └── a2a_protocol.py      # A2A message format (406 lines)
├── mcp_server/              # MCP tool implementations
│   └── tools/
│       ├── doc_search.py    # RAG tool (550 lines)
│       ├── code_run.py      # Sandbox execution (238 lines)
│       ├── quiz_bank.py     # Assessment tool
│       └── logging_tool.py  # Structured logging
├── memory/                  # State management
│   ├── session_store.py     # Session persistence
│   └── long_term_memory.py  # Learner profiles
├── observability/           # Monitoring & metrics
│   ├── audit_logger.py      # Audit trail
│   └── metrics.py           # Quality metrics
├── config/                  # Configuration
│   └── settings.py          # Global settings
├── data/                    # Data storage
│   ├── course_materials/    # Knowledge base
│   ├── chroma_db/           # Vector embeddings
│   ├── learner_profiles/    # User profiles
│   └── conversations/       # Chat history
└── main.py                  # Entry point
```

7.3 Agent Implementation Details

7.3.1 Base Agent Class

File: **agents/base_agent.py**
The **BaseAgent** class provides foundational functionality for all agents:

```
class BaseAgent(ABC):
    """Abstract base class for all agents."""

    def __init__(self, config: AgentConfig):
        self.config = config
        self.agent_id = config.agent_id
        self.capabilities = config.capabilities
        self.allowed_tools = set(config.allowed_tools)

        # Initialize Gemini LLM
        self._init_llm()

        # Register with A2A router
        a2a_router.register_agent(self.agent_id, self.capabilities)

        # Conversation history
        self._chat_history: List[types.Content] = []
```

Key Methods:

Method	Purpose
_init_llm()	Initialize Google Gemini client with API key
generate_response()	Generate LLM response with context
send_message()	Send A2A message to another agent
process_message()	Abstract method for handling incoming messages
invoke_tool()	Call MCP tool with permission checking
reset_conversation()	Clear chat history for new session

3.2 Agent Specifications

Agent	Capabilities	Allowed Tools	Primary Function
Tutor Agent	answer_question, explain_concept	logging	Session management, user interface

Agent	Capabilities	Allowed Tools	Primary Function
Knowledge Retrieval Agent	explain_concept, answer_question, document_retrieval	doc_search, logging	RAG orchestration, A2A routing
Teacher Agent	explain_concept, generate_example, ai_content, ml_content, dl_content, programming	doc_search, logging	Content generation
Code Execution Agent	code_execution, programming	code_run, logging	Sandboxed code execution
Evaluator Agent	content_evaluation, quiz_generation	doc_search, quiz_bank, logging	Assessment & feedback
Safety Agent	safety_check	logging	Policy enforcement, guardrails
Student Agent	learner_modeling	logging	Personalization, learner profiles

7.4 A2A Protocol Implementation

7.4.1 Message Types

File: **protocols/a2a_protocol.py**

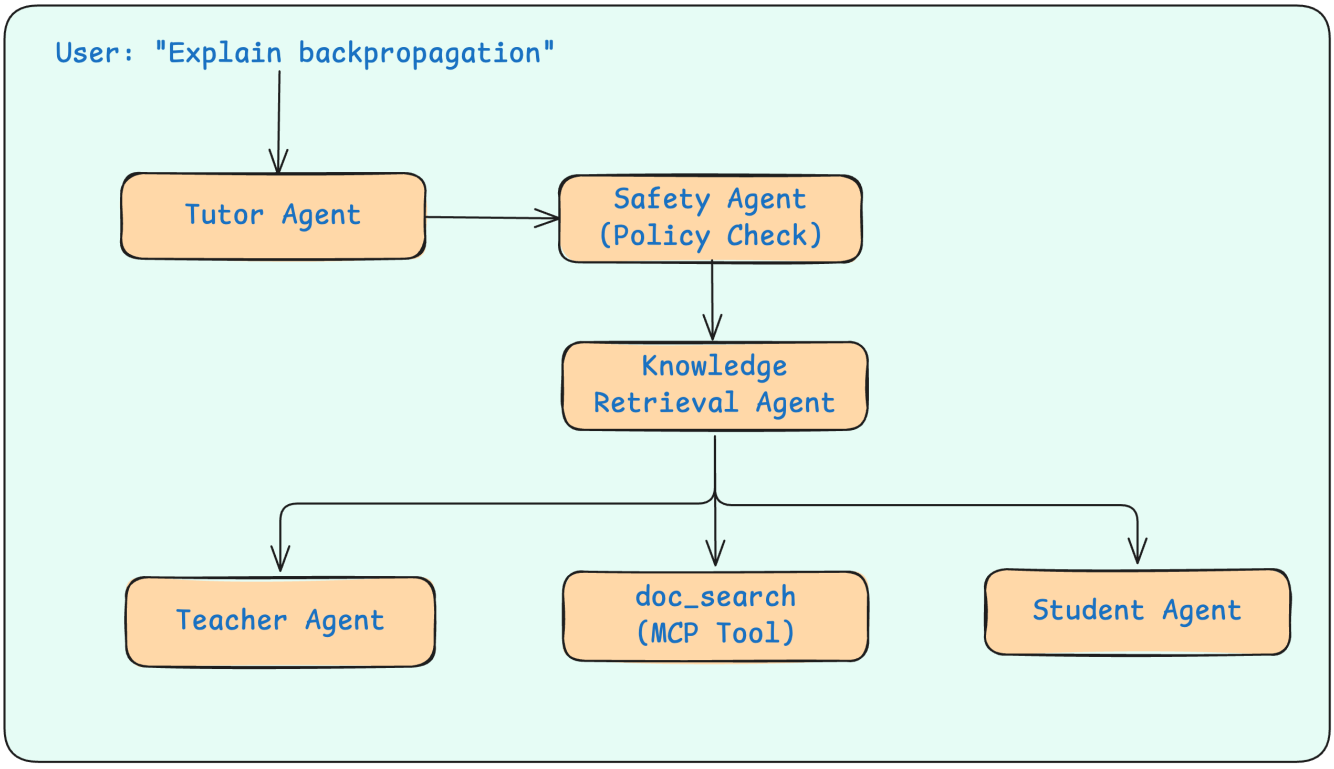
```
class MessageType(str, Enum):
    # Task-related messages
    TASK_REQUEST = "task_request"
    TASK_RESPONSE = "task_response"
    TASK_DELEGATION = "task_delegation"

    # Control messages
    CAPABILITY_QUERY = "capability_query"
    CAPABILITY_RESPONSE = "capability_response"

    # Consent and safety messages
    CONSENT_REQUEST = "consent_request"
    CONSENT_RESPONSE = "consent_response"

    # Quality control messages
    REVIEW_REQUEST = "review_request"
    REVIEW_RESPONSE = "review_response"
```

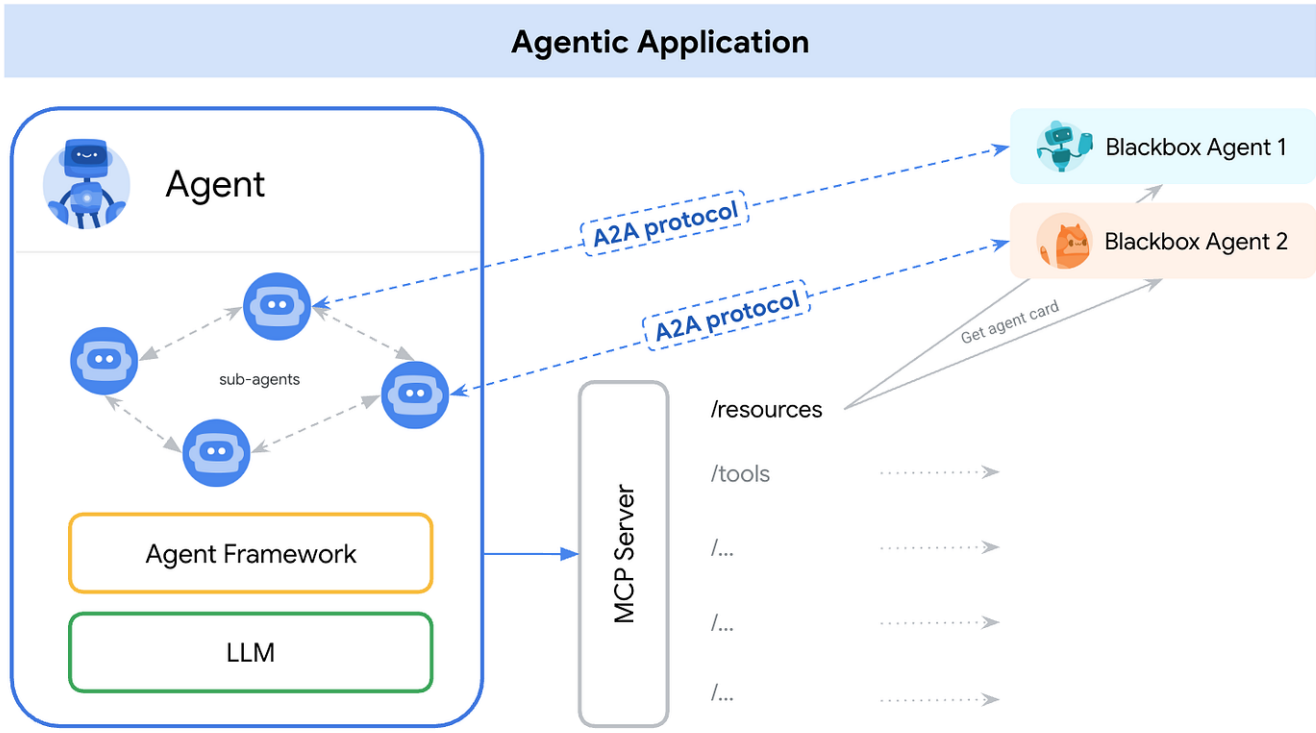
```
    REVISION_REQUEST = "revision_request"

    # Error handling
    ERROR = "error"
```

7.4.2 A2A Message Structure

```
class A2AMessage(BaseModel):
    message_id: str          # Unique identifier
    sender: str              # Source agent ID
    receiver: str            # Target agent ID
    message_type: MessageType
    payload: Dict[str, Any]  # Message content
    timestamp: datetime
    correlation_id: str      # For request-response tracking
    metadata: Dict[str, Any] # Additional context
```

7.4.3 Message Flow Example





7.5 MCP Tools Implementation

7.5.1 Document Search Tool (RAG)

File: mcp_server/tools/doc_search.py
Purpose: Semantic search over course materials using vector similarity.

Components:

ChromaDB: Persistent vector database for document embeddings
Sentence-Transformers: all-MiniLM-L6-v2 model for text embeddings

Implementation:

```
class DocSearchTool:
    TOOL_NAME = "doc_search"
    EMBEDDING_MODEL = "all-MiniLM-L6-v2"
    CHUNK_SIZE = 500
    CHUNK_OVERLAP = 100

    async def search(self, query: str, top_k: int = 5) -> SearchResult:
        # 1. Encode query using sentence-transformers
        query_embedding = self._embedding_model.encode(query)

        # 2. Query ChromaDB for similar documents
        results = self._collection.query(
```

```
        query_embeddings=[query_embedding.tolist()],
        n_results=top_k
    )

    # 3. Return ranked results with citations
    return SearchResult(chunks=chunks, search_method="semantic")
        # Simple keyword matching (fallback when semantic embeddings unavailable)
```

Chunking Strategy:

- Chunk size: 500 characters
- Overlap: 100 characters (for context continuity)
- Metadata preserved: source file, page number

7.5.2 Code Execution Tool

File: mcp_server/tools/code_run.py
Purpose: Safe Python code execution for programming practice.

Security Measures:

Measure	Implementation
AST Validation	Check for dangerous patterns (imports, file access)
Restricted Builtins	Only safe functions available
Timeout Enforcement	Prevents infinite loops (default: 10s)
No Dangerous Imports	Blocks os, sys, subprocess, etc.
Consent Requirement	User must approve before execution

```
class CodeRunTool:
    BLOCKED_IMPORTS = {'os', 'sys', 'subprocess', 'socket', 'requests'}
    BLOCKED_BUILTINS = {'exec', 'eval', 'compile', '__import__', 'open'}

    async def execute(self, code: str, consent_verified: bool) -> ExecutionResult:
        if not consent_verified:
            return ExecutionResult(success=False, error_message="Consent required")

        # 1. AST validation
        self._validate_ast(code)
```

```
        # 2. Execute in restricted environment
        result = self._run_sandboxed(code)

        return result
```

7.5.3 Quiz Bank Tool

File: mcp_server/tools/quiz_bank.py
Purpose: Generate and manage assessment questions.

Features:

- Topic-based question retrieval
- Difficulty levels (beginner, intermediate, advanced)
- Multiple question types (MCQ, short answer, coding)
- Pre-loaded question bank with 6+ questions per topic

7.6 Memory & Persistence

7.6.1 Session Store

File: memory/session_store.py
Purpose: Manage active tutoring sessions.

```
class Session(BaseModel):
    session_id: str
    user_id: str
    created_at: datetime
    last_activity: datetime
    history: List[ConversationTurn]  # Conversation history
    context: Dict[str, Any]          # Session context
    topics: List[str]                # Topics discussed
    status: str                      # active, paused, ended
```

Persistence:

- Conversations saved to: data/conversations/{user_id}_{date}_{session_id}.json
- Auto-save on each message

7.6.2 Long-Term Memory

File: memory/long_term_memory.py
Purpose: Maintain learner profiles across sessions.

Stored Data:

- Learning preferences
- Topic mastery levels
- Interaction history
- Performance metrics

Storage: data/learner_profiles/{user_id}.json

7.7 Safety Implementation

7.7.1 Safety Agent

File: agents/safety_agent.py

Defence-in-Depth Security:

Layer	Implementation
Policy Checks	Validate all requests against safety policies
Consent Workflow	HITL (Human-in-the-Loop) for sensitive operations
Prompt Injection Detection	Pattern matching for injection attempts
Content Filtering	Block harmful/inappropriate content
Rate Limiting	Prevent abuse (requests per minute)

```
class SafetyAgent(BaseAgent):
    HARMFUL_PATTERNS = [
        r"ignore.*instructions",
        r"pretend.*you.*are",
        r"bypass.*safety",
        # ... more patterns
    ]

    def _check_prompt_injection(self, text: str) -> bool:
        for pattern in self.HARMFUL_PATTERNS:
            if re.search(pattern, text, re.IGNORECASE):
```

```
            return True
        return False
```

7.7.2 Consent Requirements

Operations requiring user consent:

- Code execution
- External API calls
- Data export
- Profile modifications

7.8 LLM Integration

7.8.1 Google Gemini Configuration

File: config/settings.py

```
class GeminiConfig(BaseModel):
    api_key: str = Field(default_factory=lambda: os.getenv("GEMINI_API_KEY"))
    model_name: str = "gemini-2.0-flash"
    temperature: float = 0.7
    max_output_tokens: int = 2048
    top_p: float = 0.95
    top_k: int = 40
```

7.8.2 LLM Client Initialization

File: agents/base_agent.py

```
def _init_llm(self) -> None:
    # Initialize client with API key
    self.client = genai.Client(api_key=settings.gemini.api_key)

    # Store model name
    self.model_name = settings.gemini.model_name

    # Build generation config with system instruction
    self.generation_config = types.GenerateContentConfig(
```

```
        temperature=self.config.temperature or settings.gemini.temperature,
        max_output_tokens=self.config.max_output_tokens,
        top_p=settings.gemini.top_p,
        top_k=settings.gemini.top_k,
        system_instruction=system_prompt
    )
```

7.9 Observability & Metrics

7.9.1 Metrics Collection

File: observability/metrics.py

Metrics Categories:

Category	Metrics
Effectiveness	Task completion rate, accuracy
Efficiency	Response time, token usage
Robustness	Error rates, recovery success
Safety	Policy compliance, blocked requests

7.9.2 Audit Logging

File: observability/audit_logger.py

All agent interactions are logged with:

- Timestamp
- Agent IDs (sender/receiver)
- Message type
- Payload summary
- Outcome (success/failure)

Storage: data/metrics/metrics_{date}_{hour}.json

7.10 Data Flow Summary

10.1 Query Processing Flow

```
1. User Input → Tutor Agent
2. Tutor Agent → Safety Agent (policy check)
3. Safety Agent → Tutor Agent (approved/denied)
4. Tutor Agent → Knowledge Retrieval Agent (if approved)
5. Knowledge Retrieval Agent:
   a. → doc_search tool (RAG retrieval)
   b. → Teacher Agent (content generation)
   c. → Student Agent (personalization)
6. Knowledge Retrieval Agent → Evaluator Agent (quality check)
7. Evaluator Agent → Knowledge Retrieval Agent (feedback)
8. Knowledge Retrieval Agent → Tutor Agent (final response)
9. Tutor Agent → User (display response)
```

10.2 Code Execution Flow

```
1. User: "/code print('hello')"
2. Tutor Agent → Code Execution Agent
3. Code Execution Agent → Safety Agent (consent request)
4. Safety Agent → User (consent prompt)
5. User → Safety Agent (consent granted)
6. Safety Agent → Code Execution Agent (approved)
7. Code Execution Agent → code_run tool (sandboxed execution)
8. code_run tool → Code Execution Agent (result)
9. Code Execution Agent → Tutor Agent (formatted output)
10. Tutor Agent → User (display result)
```

7.11 Key Implementation Highlights

```
# Teacher Agent - needs doc_search for content
allowed_tools=["doc_search", "logging"]

# Code Execution Agent - needs code_run for execution
allowed_tools=["code_run", "logging"]

# Safety Agent - only logging (no data access)
allowed_tools=["logging"]
```

7.11.1 Least-Privilege Tool Access

Each agent only has access to tools required for its function:

7.11.2 Capability-Based Routing

A2A router selects agents based on advertised capabilities:

```
def route_by_capability(capability: AgentCapability) -> List[str]:
    """Find agents that can handle a capability."""
    return [
        agent_id for agent_id, caps in self._registry.items()
        if capability in caps
    ]
```

7.11.3 Conversation Persistence

Automatic saving of all conversations:

```
def _save_conversation(self, session: Session) -> None:
    filename = f"{session.user_id}_{date}_{session.session_id[:8]}.json"
    filepath = self.conversations_dir / filename
    with open(filepath, 'w') as f:
        json.dump(session.model_dump(), f, default=str)
```

7.12 Deployment

7.12.1 Installation

```
cd edusmartlearn

# Delete old Chroma data to reindex
rm -rf data/chroma_db/

# Delete all generated data
rm -rf data/conversations/
rm -rf data/learner_profiles/
rm -rf data/metrics/
rm -rf logs/

# Remove old virtual environmen (venv)
rm -rf venv/

# /opt/homebrew/bin/python3 --version
# /usr/local/bin/python3 --version
/usr/local/bin/python3 -m venv venv # please use a different path in your own machine, I have used my own python path
```

```
source venv/bin/activate  # On Windows: venv\Scripts\activate
python –version

# Install dependencies
pip install -r requirements.txt

# Configure environment from aistudio
export GEMINI_API_KEY="your-api-key"
```

12.2 Running the System

```
# Step 1: Run your test session
python main.py
# Run all 20 test queries from the queries.md file
# Type /quit when done

# Step 2: Extract metrics
You: /metrics
or
cat data/metrics/metrics_*.json
or
python extract_metrics.py
```

```
# Programmatic usage (if we want to use this for enterprise uses)
from main import EduSmartLearn
system = EduSmartLearn()
await system.initialize()
response = await system.chat("Explain neural networks")
```

Conclusion

EduSmartLearn demonstrates a production-ready implementation of:
Multi-Agent Architecture: 7 specialized agents with clear responsibilities
A2A Protocol: Type-safe, auditable inter-agent communication
MCP Tools: Standardized tool access with security controls
RAG Integration: Semantic search using ChromaDB + Sentence-Transformers
Safety-First Design: Defense-in-depth with consent workflows
Observability: Comprehensive logging and metrics collection

The system provides a solid foundation for building safe, effective AI tutoring applications.

Publications

At the time of submission, no peer-reviewed publications have been produced. The following technical articles and project notes were prepared as part of research and background study:

A2A vs MCP: Comparing Google's Agent-to-Agent Protocol with Anthropic's Model Context Protocol: https://medium.com/@saha.soumyadeep90/a2a-vs-mcp-comparing-googles-agent-to-agent-protocol-with-openai-s-model-context-protocol-6798491cc87e

Agent2Agent Protocol: Building the Language of AI Collaboration: https://medium.com/@saha.soumyadeep90/understanding-the-agent2agent-protocol-the-future-of-autonomous-system-communication-cbfa51bff502

Search, Retrieve, Respond: Model Context Protocol (MCP): https://medium.com/@saha.soumyadeep90/search-retrieve-respond-model-context-protocol-with-elasticsearch-010d60f08e26

RAG And Vector Stores Explained Simply And With A Practical Guide: https://medium.com/@saha.soumyadeep90/vector-stores-positional-encoding-and-rag-explained-simply-and-with-a-practical-guide-dea70512f6fc

Positional Encoding Explained Simply: https://medium.com/@saha.soumyadeep90/positional-encoding-explained-simply-9c6b88b5d8ff

Designing Scalable RAG Systems Using VectorDB: A Hands-On Walkthrough with ChromaDB: https://medium.com/@saha.soumyadeep90/designing-scalable-rag-systems-using-vectordb-a-hands-on-walkthrough-de9f1eac768d

Embeddings Explained: From Sparse Representations to Transformer-Based Semantic Spaces: https://medium.com/@saha.soumyadeep90/embeddings-explained-from-sparse-representations-to-transformer-based-semantic-spaces-4defcf1d78df

You need to have a google Kaggle account to access this

My Kaggle competition documentations: https://www.kaggle.com/competitions/agents-intensive-capstone-project/writeups/a-multi-agent-teaching-system-for-ai-ml-dl-and-pro

My Kaggle competition working code: https://www.kaggle.com/code/soumyadeepsaha999/google-soumyadeep-capstone-project

Appendix (if any)

Appendix A: Example A2A Message Schema (Simplified)

A2A messages are structured objects exchanged between agents. They include identity fields, capability information, consent metadata, and payload. A simpified task request example is shown below.

```
{
 "message_id": "msg_001",
 "timestamp": "2026-01-28T10:15:00Z",
 "message_type": "task_request",
 "sender": {"agent_id": "orchestrator"},
 "receiver": {"agent_id": "code_execution_agent"},
 "capability": "execute_and_debug_code",
 "consent": {"required": true, "status": "pending"},
 "payload": {
  "language": "python",
  "code": "print(sum(range(10)))",
  "constraints": {"timeout_s": 5, "network": "disabled"}
 },
 "trace": {"session_id": "sess_123", "span_id": "span_abc"}
}
```

Protocol Benefits:

- Clear separation of concerns between agents
- Capability negotiation before tool usage
- Explicit consent mechanisms for sensitive operations
- Structured error reporting and handling
- Full auditability of all inter-agent communications

A corresponding response message includes tool outputs and an explicit status/error field:

```
{
 "message_id": "msg_002",
 "timestamp": "2026-01-28T10:15:03Z",
 "message_type": "task_response",
 "sender": {"agent_id": "code_execution_agent"},
 "receiver": {"agent_id": "orchestrator"},
 "status": "success",
 "payload": {
  "stdout": "some topics generated",
  "stderr": "",
  "exit_code": 0
 },
 "error": null,
```

```
  "trace": {"session_id": "sess_123", "span_id": "span_def"}
}
```

<u>Appendix B:</u> Example Agent Card (Capability Advertisement)

An agent card is a machine-readable description of an agent's capabilities and interfaces. It supports discoverability and safer routing decisions by the orchestrator.

```
{
  "agent_id": "retrieval_agent",
  "display_name": "Retrieval Agent",
  "version": "0.1",
  "capabilities": [
    {"name": "doc_search", "description": "Retrieve and summarize trusted course material."}
  ],
  "mcp_tools": ["doc_search"],
  "policy_tags": ["read_only", "no_code_execution"],
  "endpoint": "a2a://edusmartlearn/retrieval_agent"
}
```

<u>Appendix C:</u> Example MCP Tool Schema (Code Execution)

MCP tools are exposed with schemas that define parameters and expected outputs. A simplified schema for a sandboxed code execution tool is shown below.

```
{
  "tool_name": "code_run",
  "description": "Execute code in a sandbox with resource limits.",
  "input_schema": {
    "type": "object",
    "properties": {
      "language": {"type": "string", "enum": ["python", "java"]},
      "code": {"type": "string"},
      "timeout_s": {"type": "integer", "minimum": 1, "maximum": 30}
    },
    "required": ["language", "code"]
  },
  "output_schema": {
    "type": "object",
    "properties": {
      "stdout": {"type": "string"},
      "stderr": {"type": "string"},
      "exit_code": {"type": "integer"}
    },
    "required": ["stdout", "stderr", "exit_code"]
```

}
}

<u>Appendix D:</u> Consent Workflow for Sensitive Tool Calls

For sensitive actions (example: executing code or storing memory), the system uses an explicit consent workflow:

- Agent proposes an action and marks the request as consent_required in the A2A message.
- Safety/Policy Engine checks whether the request is within policy and within the agent's scope.
- Tutor Agent asks the user for approval (approve/deny) in natural language trigger or UI.
- On approval, the MCP tool call is executed; on denial, the system provides a safe alternative (e.g., explanation without execution).
- All decisions (proposed action, policy result, user decision) are recorded in the audit log.

References

Read the "Introduction to Agents" whitepaper: https://www.kaggle.com/whitepaper-introduction-to-agents

Agent Tools & Interoperability with Model Context Protocol (MCP)
Read the "Agent Tools & Interoperability with Model Context Protocol (MCP)" whitepaper: https://www.kaggle.com/whitepaper-agent-tools-and-interoperability-with-mcp

Context Engineering: Sessions & Memory
Read the Context Engineering: Sessions & Memory whitepaper: https://www.kaggle.com/whitepaper-context-engineering-sessions-and-memory

Agent Quality
Read the Agent Quality whitepaper: https://www.kaggle.com/whitepaper-agent-quality

Prototype to Production
Read the Prototype to Production whitepaper: https://www.kaggle.com/whitepaper-prototype-to-production

Adaptation of Agentic AI: https://arxiv.org/pdf/2512.16301

Recording Demo
Link: https://www.loom.com/share/a0f621e993bd41b0bb020a0ced91228e

GitHub Code commit link: https://github.com/soumyadeep-saha/M25AI1015_MTP_EduSmartLearn/tree/main